

Assignment 5: chArm-v3 System Emulator

C S 429, Spring 2025

Unique ID: 50815, 50820, 50825, 50830, 50835, 50840, 50845, 50850, 50855, 50860, 50865, 50870

Lead TAs: Kiran Chandrasekhar, Kavya Rathod, Wyatt Borden

Assigned: Friday, 29 March 2025 23:00 CT

Checkpoint 1 submission: Thursday, 3 April 2025 23:59 CT

Checkpoint 2 submission: Thursday, 10 April 2025 23:59 CT

Checkpoint 3 submission: Thursday, 17 April 2025 23:59 CT

Checkpoint 4 submission: Thursday, 24 April 2025 23:59 CT

Last possible hand-in: Saturday, 26 April 2025 23:59 CT

1 Introduction

In this lab, you will be implementing several simulators:

- `hw_elts`, a set of individual hardware elements emulating hardware, including a register file for writing/reading register values and an ALU (Arithmetic Logical Unit) to perform computations;
- `psim`, a standalone simulator for the PIPE implementation of the chArm-v3 instruction set architecture (ISA), assuming an ideal memory system;
- `csim`, a standalone trace-driven cache simulator for a simple two-level memory hierarchy; and
- `pcsim`, an integrated “PIPE-with-CACHE” simulator, by enhancing `psim` to handle variable delays in the memory stage, and connecting it to `csim`, resulting in a simulator for the PIPE implementation of the chArm-v3 ISA with a simple two-level memory hierarchy.

The common executable that you will create will be named `se`. You will be working on the `hw_elts` and `psim` aspects of it in the first two checkpoints, upgrading the `psim` in the third checkpoint, and working on the `csim` and `pcsim` aspects in the fourth checkpoint.

Outcomes you will gain from this lab include the following:

- You will understand how the SEQ and PIPE- implementations of chArm-v3 work. You will understand the function of each stage and how they are connected to each other.
- You will understand how the PIPE implementation of chArm-v3 works. You will understand how stalling, squashing, and forwarding help resolve different hazard conditions.
- You will understand the impact that cache memories can have on the performance of programs.
- You will understand the additional changes that need to be made to the PIPE implementation to accommodate a (semi-)realistic memory hierarchy.

2 Logistics

This assignment lasts for four weeks, with four separate submissions. You are required to perform four submissions:

- HW_ELTS, due by Thursday, 3 April 2025 23:59 CT.
- PIPE-, due by Thursday, 10 April 2025 23:59 CT.
- PIPE, due by Thursday, 17 April 2025 23:59 CT.
- CSIM and PCSIM, due by Thursday, 24 April 2025 23:59 CT.

Start early enough to get the assignment done before the due date. Assume things will not go according to plan, and so you must allow extra time for heavily loaded systems, dropped internet connections, corrupted files, traffic delays, minor health problems, *force majeure*, etc.

This is an individual or partner project. If you choose to work in pairs, the team may use as many slip days as the partner with the fewest available slip days. That is, if you have two slip days and your partner has three, the team gets two slip days to use for the entire assignment. Both partners will be charged for any slip days used. Choose your partner well; if you and your partner decide to split, both of you will have to do the lab alone for all remaining checkpoints. Partners on a team do not have to be from the same section of the class.

Up to 2 slip days can be used per checkpoint, but remember that using slip days for a checkpoint does not adjust any future due dates. **Remember also that no slip days are allowed for Checkpoint 1.**

All hand-ins are electronic. You may do your coding on any machine you choose, *but it is your responsibility to test this assignment for correct build/execution on an UTCS 64-bit x86-64 Linux machine before your final hand-in.* You may not share your work on lab assignments with other students outside your team, but feel free to ask instructors for help (e.g., during office hours or discussion sections). Unless it is an implementation-specific question (i.e., private to instructors), please post it on Ed Discussion publicly and non-anonymously so that students with similar questions can benefit as well.

Before you begin, please take the time to review the course policy on academic integrity at: <https://www.cs.utexas.edu/academics/conduct>. Don't copy code from anywhere; do it yourself. This discipline is very important for this class and next classes.

Any updates for this lab will be posted on Canvas. Any clarifications or corrections for this lab will be posted on Ed Discussion.

3 Download and Setup

Go to the GitHub Classroom assignment page found at: <https://classroom.github.com/a/5JL6HKRK>. You will be able to clone your assignment to a lab machine as usual. Also note that since you can work in pairs, GitHub Classroom may prompt you to enter who you are working with to set up your repository with them. You will only need one repository for submitting, but if both you and your partner make one, just decide whose to submit later on.

4 Assignment Details

4.1 Notion

This *Wiki page* is a work in progress page created by the TAs to help guide you throughout the lab. It will be updated on an ongoing basis through the assignment. There is a lot of information here! **Please be sure to read the notion BEFORE making Ed posts for notion-answerable questions!**

4.2 Repository Structure

Now that you have your private repository of the code base, confirm that you have the following subdirectories within it.

- **include:** This subdirectory contains all the header files needed for this project. It contains three subdirectories: `pipe`, which contains the header files needed for the pipelined processor implementation; `cache`, which contains the header files needed for the cache simulator; and `base`, which contains the header files needed for the underlying simulator.
- **src:** This subdirectory contains all the source code files needed for this project. It contains three subdirectories: `pipe`, which contains the source code files needed for the pipelined processor implementation; `cache`, which contains the source code files needed for the cache simulator; and `base`, which contains the source code files needed for the underlying simulator.

To complete the hardware elements in `hw_elts`, you will modify the file `src/base/hw_elts.c`.

To complete the pipelined processor emulator `psim` in PIPE, you will modify the following files, all of which are located in the `src/pipe` directory:

`instr_Fetch.c`, `instr_Decode.c`, `instr_Execute.c`, `instr_Memory.c`,
`instr_Writeback.c`, `forward.c`, and `hazard_control.c`.

To complete the standalone cache simulator `csim` for the CSIM deliverable, you will modify only the file `src/cache/cache.c`.

To complete the final `pcsim`, you will further modify only these files.

- **testcases:** This subdirectory contains test cases for testing your emulator. The subdirectories `basics`, `alu`, `mem`, `branch`, `exceptions`, and `applications` contain assembly `.s` files, disassembled `.od` files, and AArch64/Linux binaries that are used to test your simulator. Some of these directories are further subdivided into `pipeminus`, `hazard`, and `hard`, which narrow the focus of the tests. Finally, the `cache` subdirectory contains memory trace files similar to those that you used in MM Lab, which are used to test your cache simulator.

4.3 Simulator Quirks

There are a few things in the simulator that you would not find in normal hardware, so we would like to note them here.

First, it is important to note that `hlt` is a privileged instruction, and `gcc` won't typically compile files with that instruction. To shut down the emulator, we instead check for a `ret` instruction with a return address of 0, which we turn into an emulated HLT instruction that stops the simulator after it reaches the writeback stage. This functionality is implemented for you in `src/pipe/instr_Fetch.c`, and you only need to pass the generated `STAT_HLT` through the pipeline for it to work.

Second, we need ways to view the state of the machine during a program's execution for our testing purposes, so we have special "trap" addresses defined for this purpose. In `src/base/mem.c`, you can find `IO_CHAR_ADDR` and `CHECKPOINT_ADDR` defined. A load from `CHECKPOINT_ADDR` will log the machine state to a provided "checkpoint file" (more on this later), and a store to `IO_CHAR_ADDR` will print the stored register's contents to the terminal.

5 Programming Tasks

This lab is a sequence of several programming parts. In Checkpoint 1, you will implement `hw_elts`, an architecture element emulator (ALU, register file, etc.). In Checkpoint 2, you will implement `psim`, a PIPE- simulator. In Checkpoint 3, you will augment `psim` to implement PIPE. In Checkpoint 4, you will implement `pcsim`, a "PIPE-with-CACHE" simulator, by implementing a cache simulator `csim`, enhancing `psim` to handle variable delays in the memory stage, and connecting them to each other.

5.1 Checkpoint 1: Implementing `hw_elts`, The Hardware Elements of a chARM-v3 CPU

Start by implementing the hardware elements in the `hw_elts.c` file. These emulate modules found in the hardware that will be called upon in your pipeline. Once you do these correctly, you will be able to pass all the tests in the `bin/test-hw` testbench.

You are required to implement three functions that emulate three hardware modules for your HW simulator:

- `cond_holds()`: This module determines if a given condition code is met based on the NZCV flag state. This should not be directly invoked from your pipeline emulator.
- `alu`: This module computes bitwise and mathematical operations like addition or shifts, and sets the NZCV flags accordingly. This module will also invoke the `cond_holds()` module to set a condition value.
- `regfile`: This module will allow you to simultaneously read and write to registers. Take care to handle writes to and reads from the SP and XZR registers properly, which are not general-purpose.

HW will need to be complete for the PIPE- implementation to function properly. If in doubt, read the ARM manual linked on the syllabus page.

5.1.1 Testing HW

Make your testbench `test-hw` in the usual manner: `make clean; make`. There is no standalone hw executable. The standard set of make targets that we have been using all semester long are also used in this assignment. Run your `test-hw` executable with the `-h` using the command line to see the usage of the executable.

Debug your program in the normal manner using `gdb` directly on the testbench.

5.2 Checkpoint 2: Basic Pipelined Implementation (PIPE-)

Implement the simpler PIPE- implementation we have discussed in class. If you do this correctly, you will be able to pass all the tests in the `testcases/basics` and `testcases/*/pipeminus` directories.

You are required to implement five functions that emulate five stages for your PIPE- simulator:

- `fetch_instr()`: Fetch stage (including PC update actions).
- `decode_instr()`: Decode stage.
- `execute_instr()`: Execute stage.
- `memory_instr()`: Memory stage.
- `wback_instr()`: Write-back stage.

There are a number of other functions in these files that correspond to the various logic blocks discussed in class. You must implement these functions as well, and then “wire them up” appropriately within the stage logic.

The fields of the pipeline registers are defined in `include/pipe/instr_pipeline.h` in several struct types `*_instr_impl_t` and `pipe_reg_t`. The “clocking” of these registers is handled for you in `src/base/proc.c`. When implementing the combinational logic for a pipeline stage, you will be passed in the appropriate structs as your input and output arguments, which are named `in` and `out`. There are also a few global variables that represent the “backwards” wires sent from one stage to a prior stage, which you will need to update as well.

5.2.1 Testing

Make your `se` executable in the usual manner: `make clean; make`. The standard set of `make` targets that we have been using all semester long are also used in this assignment.

You can also do `make week<week number>` for individual weeks.

To test individual testcases, use the `se` executable. To run all of the testcases for the week and compare them to the reference output use the `test-se` executable. To see what flags are necessary and what each one means, both binaries have a `-h` flag that will tell you what each flag does.

Debug your program in the normal manner using `gdb`. There is also a provided `se-ref` executable for each checkpoint that you can use (with the same arguments as `se`) to check the correct output of a test case. Or you can run an exhaustive test with `test-se`.

5.3 Checkpoint 3: Hazard Control

For the third checkpoint, you are also required to implement stalling, squashing, and forwarding to deal with data hazards and control hazards, as discussed in class. For forwarding, add your implementation in `src/pipe/forward.c` and then call the resulting function(s) from `src/pipe/instr_Decompile.c`. For stalling, implement the four functions in `src/pipe/hazard_control.c`, which is called for you in `src/base/proc.c`. You are responsible for setting up the correct `stall` and `bubble` signals for each pipeline stage at each cycle. The task of taking the appropriate actions on the pipeline registers based on these signals is handled for you.

5.3.1 Testing

Testing for Checkpoint 3 is nearly identical to Checkpoint 2. Something changes in how you invoke `test-se`, use the `-h` flag to figure out what to change.

5.4 Checkpoint 4: Cache, Putting it all together

In this part, you will first write a standalone cache controller simulator `csim` and test it against a number of memory traces. Correctness will be determined by matching the cache events generated by your simulator against a reference. You will then augment `psim` and connect it to `csim` to produce `pcsim`.

- Start your work in the `cache` directory.
- Implement the `get_line()` and `select_line()` helper functions in the file `cache.c`. Implement the `check_hit()` and `handle_miss()` routines in the file `cache.c`. This will give you a skeletal cache simulator that implements the control actions (the three-state finite-state machine cache controller discussed in class) of a write-back cache with LRU replacement and write-allocate policies, for arbitrary numbers of sets, associativity values, and block sizes.
- You can assume that each cache read/write only accesses one single cache line.
- Implement the functions `get_word_cache()` and `set_word_cache()` in the file `cache/cache.c`. After completing this task, you will have a fully functional cache simulator that implements both the control and the data portions of the cache.
- Integrate the memory hierarchy simulator into the PIPE simulator by making the appropriate changes in the files that you updated for the earlier checkpoints. This should involve no more than handling stalls resulting from cache misses.

5.4.1 Testing

You can test your cache implementation by running `make` and running `bin/test-csim`. Your implementation is correct when the test score printed out is 40/40.

You can test your “all-together” implementation (aka `pcsim`) using `bin/test-se`, paying special attention to the `-w` flag. Several AArch64/Linux binaries will be run with a few different cache configurations. You may test these yourself by adding the command-line arguments that are associated with the cache configuration to any test from before. Figure out the arguments by reading the output from the `-h` flag.

6 Submission

Submit your checkpoint versions using Gradescope, by providing a link to the private GitHub repository where you have done your work. Remember to include your partner in your submission, if you have one.

- *Clearing Checkpoint 1* corresponds to correctly running the `bin/test-hw` testbench with full points. There is no partial credit for this portion of the project. It is imperative that your HW implementation is 100% correct in order for the rest of the lab to function correctly. It is also your responsibility to accomplish this; no default implementation will be provided to you under any circumstances.
- *Clearing Checkpoint 2* corresponds to your simulator correctly running the test programs in the `testcases/basics` and `testcases/*/pipeminus` subdirectories.
- *Clearing Checkpoint 3* corresponds to your simulator correctly running test programs in the `testcases/*/hazard` and `testcases/*/hard` subdirectories.

- *Clearing Checkpoint 4* corresponds to your simulator correctly running test programs in the `testcases/mem` and `testcases/hard` subdirectories, with cache enabled. Additionally, it requires correctly handling each trace in `testcases/cache`.

Note that “correctly running test programs” means that the checkpoint files generated by your implementation must match those generated by the reference simulator provided to you. Printing the same value to the terminal as the reference does not guarantee that your implementation is correct, though it should at least be a decent hint. The utility `bin/test-se` is provided to specify which tests correspond to which checkpoint, as well as which tests are passing.

6.1 Scoring

The assignment is worth 16 points: two points for Checkpoint 1, six points for Checkpoint 2, and four each for Checkpoints 3 and 4. Your submission will be auto-graded based on its ability to correctly execute the test cases for the corresponding week. There will also be some extra credit offered, the details (including point allocations) of which will be released soon.